

Object Detection

Object Detection

- 1. Model Introduction
- 2. Target Prediction: Image
 - Effect Preview
- 3. Target Prediction: Video
 - Effect Preview
- 4. Target Prediction: Real-time Detection
 - 4.1. USB Camera
 - Effect Preview
 - 4.2. CSI Camera
 - Effect Preview
- Reference

Use Python to demonstrate **Ultralytics**: Object Detection effects on images, videos, and real-time detection.

1. Model Introduction

Object detection is a task that involves identifying the location and category of objects in an image or video stream.

The output of an object detector is a set of bounding boxes that enclose the objects in the image, along with the class label and confidence score for each bounding box. If you need to identify objects of interest in a scene, but do not need to know the specific location or exact shape of the object, then object detection is a good choice.

2. Target Prediction: Image

Use yolo26n_ncnn_model to predict the image under the ultralytics26 folder.

Enter the code folder:

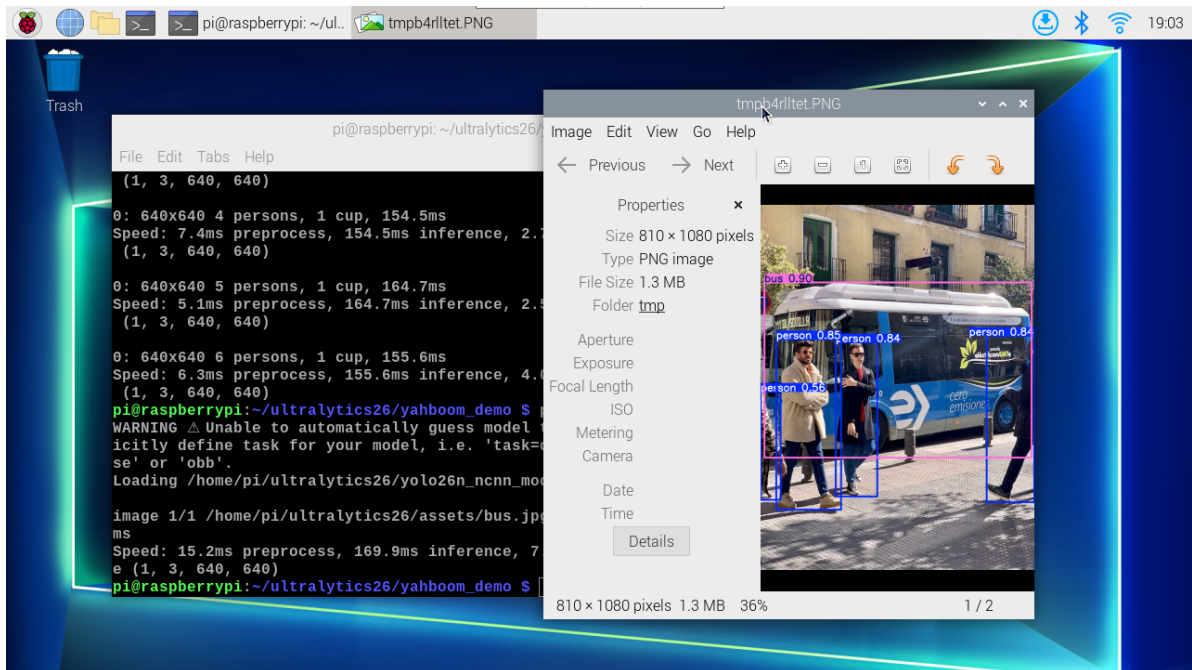
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code:

```
python3 01.detection_image.py
```

Effect Preview

The location of the image output by yolo recognition: /home/pi/ultralytics26/output/



Sample code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/pi/ultralytics26/yolo26n_ncnn_model")
# model = YOLO("/home/pi/ultralytics26/yolo26n.pt")

# Run batched inference on a list of images
results = model("/home/pi/ultralytics26/assets/bus.jpg") # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/home/pi/ultralytics26/output/bus_output.jpg") # save to disk
```

3. Target Prediction: Video

Use yolo26n_ncnn_model to predict the video under the ultralytics26 folder (not the video that comes with ultralytics).

Enter the code folder:

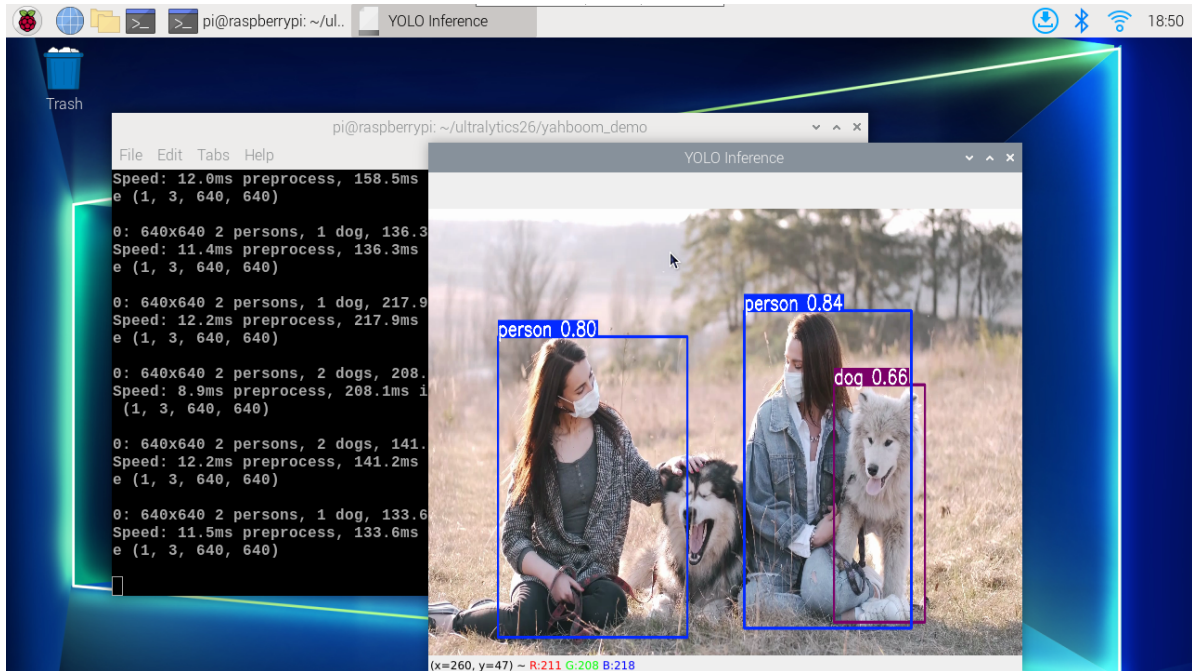
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code:

```
python3 01.detection_video.py
```

Effect Preview

The location of the video output by yolo recognition: /home/pi/ultralytics26/output/



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/pi/ultralytics26/yolo26n_ncnn_model")
# model = YOLO("/home/pi/ultralytics26/yolo26n.pt")

# Open the video file
video_path = "/home/pi/ultralytics26/videos/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/pi/ultralytics26/output/01.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
```

```

annotated_frame = results[0].plot()

# Write the annotated frame to the output video file
out.write(annotated_frame)

# Display the annotated frame
cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4. Target Prediction: Real-time Detection

4.1. USB Camera

Use yolo26n_ncnn_model to predict the USB camera frame.

Enter the code folder:

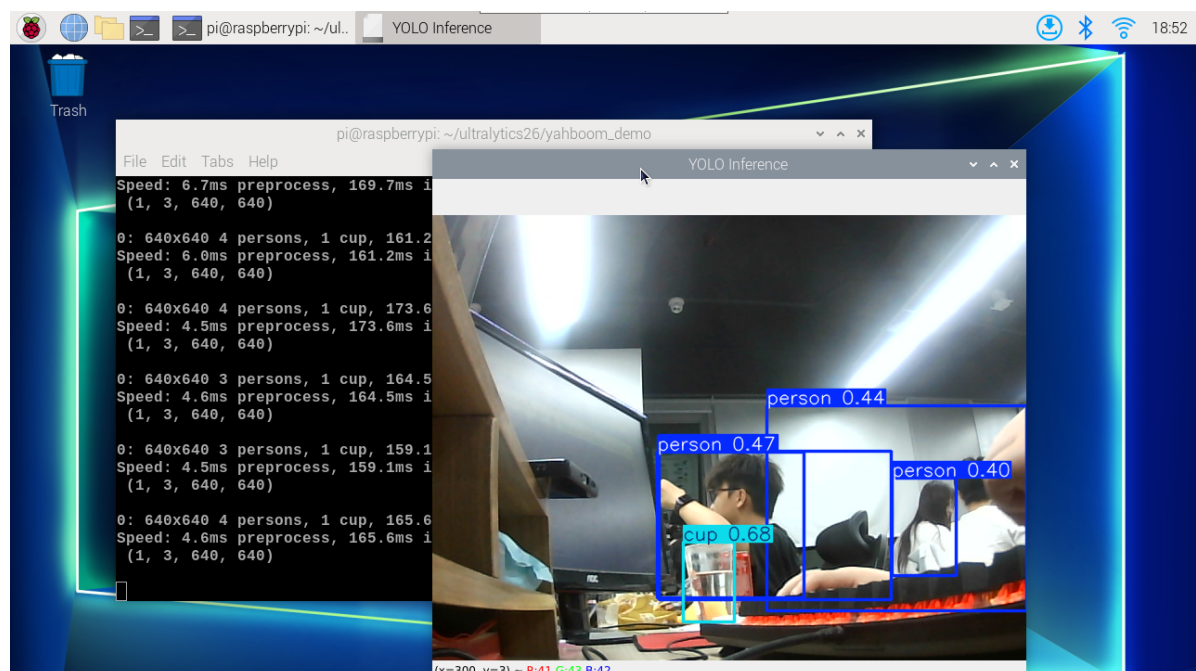
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code: Click on the preview screen and press the q key to terminate the program!

```
python3 01.detection_camera_usb.py
```

Effect Preview

The location of the video output by yolo recognition: /home/pi/ultralytics26/output/



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/pi/ultralytics26/yolo26n_ncnn_model")
# model = YOLO("/home/pi/ultralytics26/yolo26n.pt")

# Open the cammera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/pi/ultralytics26/output/01.detection_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

4.2. CSI Camera

Use yolo26n_ncnn_model to predict the CSI camera frame.

Enter the code folder:

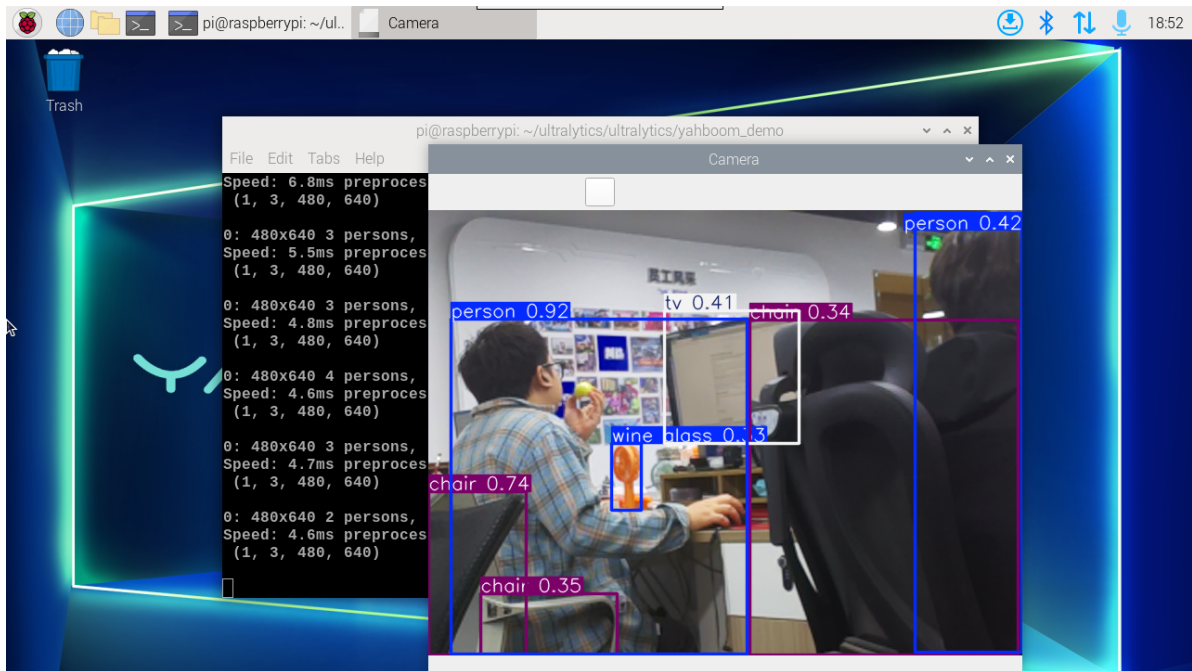
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code: Click on the preview screen and press the q key to terminate the program!

```
python3 01.detection_camera_csi.py
```

Effect Preview

The location of the video output by yolo recognition: /home/pi/ultralytics26/output/



Sample code:

```
import cv2
from picamera2 import Picamera2

from ultralytics import YOLO

# Initialize the Picamera2
picam2 = Picamera2()
picam2.preview_configuration.main.size = (640, 480)
picam2.preview_configuration.main.format = "RGB888"
picam2.preview_configuration.align()
picam2.configure("preview")
picam2.start()

# Load the YOLO11 model
model = YOLO("/home/pi/ultralytics26/yolo26n_ncnn_model")
# model = YOLO("/home/pi/ultralytics26/yolo26n.pt")

# Set up video output
output_path = "/home/pi/ultralytics26/output/01.detection_camera_csi.mp4"
```

```
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(output_path, fourcc, 30, (640, 480))

while True:
    # Capture frame-by-frame
    frame = picam2.capture_array()

    # Run YOLO inference on the frame
    results = model(frame)

    # Visualize the results on the frame
    annotated_frame = results[0].plot()

    # Write the frame to the video file
    out.write(annotated_frame)

    # Display the resulting frame
    cv2.imshow("Camera", annotated_frame)

    # Break the loop if 'q' is pressed
    if cv2.waitKey(1) == ord("q"):
        break

# Release resources and close windows
picam2.close()
out.release()
cv2.destroyAllWindows()
```

Reference

[Object Detection - Ultralytics YOLO Documentation](#)